

From Behaviors to Contextual Refinement

Program Verification Workshop 2026

Day 1 · CRIS fundamentals and guided proofs

What can this program do?

```
Check Beh.of_itree.  
(* itree coreE Any.t -> Tr.t -> Prop *)
```

Beh.of_itree program trace says that trace is one behavior of program .

| Read it as trace membership.

Return produces a terminating trace

```
Definition return_42 : itree coreE Any.t :=  
  Ret (42%Z)↑.  
  
Check ((42%Z)↑ : Any.t).  
Check (Tr.done (42%Z)↑).  
Check (Beh.of_itree return_42 (Tr.done (42%Z)↑)).
```

Ret 42 $\longrightarrow$ done 42

`Any.t` stores the whole-program return value. The notation `x↑` injects a typed value into `Any.t`.

Tau preserves the observation

```
Definition silent_return_42 : itree coreE Any.t :=  
  tau;; return_42.
```

```
tau; Ret 42  —————> done 42  
Ret 42  —————> done 42
```

Tau is an internal computation step.

Choose gives several behaviors

```
Definition choose_bit : itree coreE Any.t :=  
  'b : bool <- trigger (Choose bool);;  
  Ret ((if b then 1%Z else 0%Z)↑).
```

```
choose_bit → done 0  
choose_bit → done 1
```

One program supports both membership propositions.

I/O extends the trace

```
Definition echo_once : itree coreE Any.t :=  
  'answer : Z <- trigger (@IO Z Z "echo" 7%Z);;  
  Ret answer↑.
```

```
request echo(7)  
  ┆ response 9 → interact echo(7 ↦ 9); done 9  
  ┆ unanswered → hang echo(7)
```

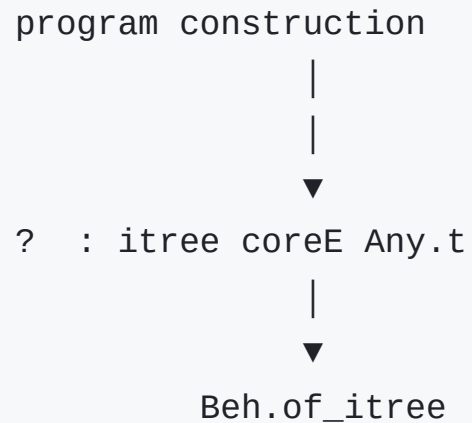
Trace shapes

```
Print Tr.t.  
Check Tr.spin.
```

```
done(ret)      abort      spin  
hang(request)  interact(io, rest)
```

- `spin` : infinitely many internal steps
- `interact` : one observable I/O followed by the remaining trace
- an infinite I/O behavior: an infinite chain of `interact`

Where does the closed interaction tree come from?



`Behavior.v` started at the final observation interface.

coreE is the residual whole-program signature

```
Print coreE.
```

```
coreE = Choose | Take | IO
```

After module compilation:

- named function calls have been resolved;
- module-local state operations have been interpreted;
- the client/service boundary has been erased.

Residual effects: **Choose** / **Take** branch; **IO** records observable interaction.

day1/lectures/Behavior.v → ModuleIntro.v

Why build the program from modules?

Modular programming and verification

- Modules preserve named component boundaries during implementation and proofs.
- A client can link against a refined service implementation.

State interpretation

- Each `SMod.t` supplies an initial local state.
- Compilation interprets `SGet` and `SPut` over the combined module state.

Two modules, one linked program

```
module ConstService { get() = 42 }  
module Main          { main() = ConstService.get() }
```

Define the two modules separately, then link them:

The diagram illustrates the process of linking two modules. On the left, 'ConstModule' and 'MainModule' are listed. Brackets from each module point to a central star symbol (★). From the star, an arrow points to the text 'whole program', which then has an arrow pointing to 'behavior'.

```
ConstModule ┌  
             │ ── ★ ──> whole program ──> behavior  
MainModule  └
```

A header names a typed boundary

```
Module ConstHdr.  
  Definition get :=  
    fnsig (fn "get") (fntyp () Z).  
End ConstHdr.
```

```
get : unit → Z
```

Callers and implementations share this signature.

`Module ConstHdr` is a Rocq namespace. `ConstHdr.get` is a concrete CRIS function signature stored inside that namespace.

`mn +:+ "." +:+ method` builds its qualified string name.

Ordinary return, interaction-tree return

Ordinary code:

```
get() = return 42
```

CRIS function body:

```
Definition get : () -> itree crisE Z :=  
  fun _ => Ret 42%Z.
```

```
Check get.
```

Ret constructs an interaction tree that returns immediately.

Inspect the function-body event signature

```
Print crisE.  
Check Call.  
Check SGet.  
Check SPut.
```

```
crisE = agE +' callE +' pgE +' coreE  
          |      |  
          Call  SGet / SPut
```

Relevant constructors today are `Call`, `SGet`, and `SPut`.

```
get : () -> itree crisE Z
```

Package the service

```
Program Definition smod : SMod.t := {|  
  SMod.scopes := scopes;  
  SMod.fnsems := fnsems;  
  SMod.initial_st :=  $\emptyset$ ;  
|}.  
  
Definition t : Mod.t := SMod.to_mod  $\emptyset$  smod.
```

module = named function bodies + private initial state + scope

This first service uses empty local state.

`smod : SMod.t` is a semantic module value. `SMod.to_mod` turns it into the `Mod.t` value consumed by linking.

Ordinary call, named CRIS call

Ordinary code:

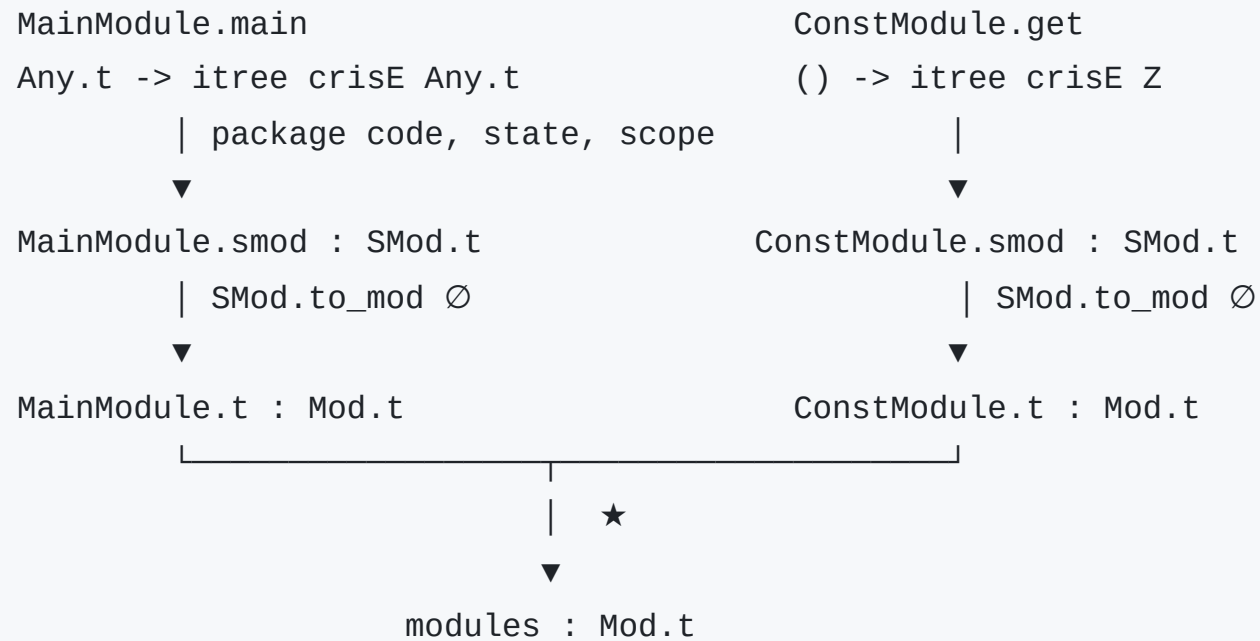
```
main() = ConstService.get()
```

CRIS function body:

```
Definition main : Any.t -> itree crisE Any.t :=  
  fun _ =>  
    n <- ccallU ConstHdr.get tt;;  
    Ret n↑.
```

`ccallU ConstHdr.get tt` emits a named `call` event.

From function fragments to a linked module



Link, compile, observe

```
Definition modules : Mod.t :=  
  MainModule.t ★ ConstModule.t.
```

```
Definition linked : LMod.t :=  
  Mod.to_lmod modules ε.
```

```
Definition program : itree coreE Any.t :=  
  LMod.compile linked tt↑.
```

```
Check (Beh.of_itree program).
```

From the linked module to behaviors

```
modules : Mod.t
  | Mod.to_lmod modules ε
  ▼
linked : LMod.t
  | LMod.compile linked tt↑
  ▼
program : itree coreE Any.t
  | Beh.of_itree
  ▼
Beh.of_itree program : Tr.t -> Prop
```

- `Mod.to_lmod` turns `SGet` and `SPut` into operations on linked state.
- `LMod.compile` resolves named calls and interprets that state, leaving `coreE`.

ModuleIntro.v → RefinementIntro.v · motivation

Linked modules may have different owners

APPLICATION TEAM

Client

calls Cell.set / Cell.get
proves an output property

LIBRARY TEAM

Cell library

publishes API + contract
owns code + private state

\ /\n deployment
 Client ★ Cell

A module boundary can also be an ownership boundary.

A Cell has public behavior and private code

PUBLIC API + CONTRACT

set : Z -> unit

get : unit -> Z

set(v) updates the logical cell to v.

With no intervening set, a following get() returns v.

ONE PRIVATE IMPLEMENTATION

state current := 0

set(v): current := v

get(): return current

The application team receives the public contract.
The library team owns the private implementation.

The client proof uses the contract

Client code

```
Cell.set(42)
x <- Cell.get()
output(x)
```

Proof from CellSpec

```
set(42)
  | contract
  ▼
get() returns 42
  |
  ▼
output(42)
```

The proof mentions `set` , `get` , their specified results, and observable I/O.

The provider may change private code

RELEASE 1

```
set(v):  
  current := v
```

```
get():  
  return current
```

RELEASE 2

```
set(v):  
  if current != v:  
    current := v
```

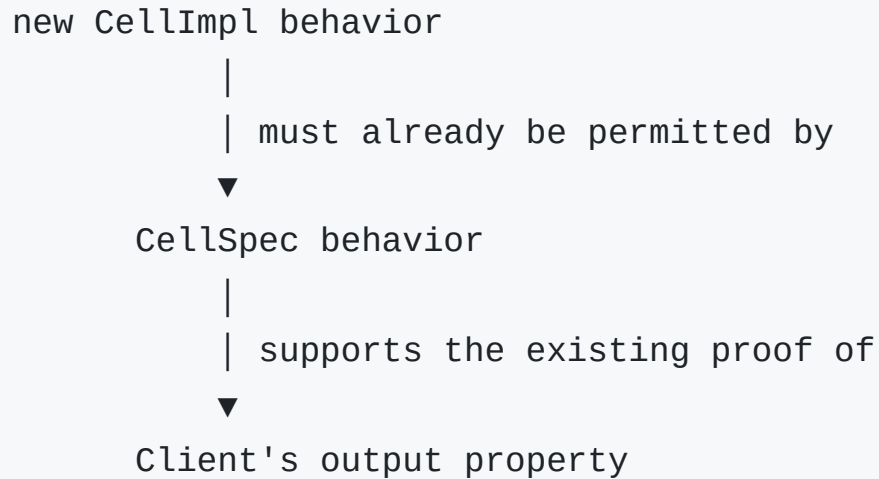
```
get():  
  return current
```

Each release must satisfy the same public Cell contract.
The client code and its output proof stay unchanged.

Preserve the client proof across releases

The application proof was derived from `CellSpec` .

Every new `CellImpl` release must stay within that contract:



A release with an extra observable behavior could invalidate the client proof.
The provider must rule out such a behavior in every client context.

One theorem validates every future client

The provider publishes the implementation before seeing every future client.
The replacement theorem therefore quantifies over a client context:

```
for every compatible Ctx,  
  Beh(CellImpl ★ Ctx) ⊆ Beh(CellSpec ★ Ctx)
```

Every deployed implementation trace is permitted by the published contract.
This all-client replacement theorem is **contextual refinement**.

Open `day1/lectures/RefinementIntro.v`.

The provider's contextual-refinement theorem

The implementation, specifications, and client remain abstract `Mod.t` values:

```
Variables (CellImpl CellSpec Client ClientSpec : Mod.t).  
  
Check (⊢ ctx_refines CellImpl CellSpec).
```

The library provider proves:

```
⊢ ctx_refines CellImpl CellSpec
```

`CellImpl` is the deployed target. `CellSpec` is the published source.

Specialize the theorem to one client

```
Lemma cell_in_one_context :  
  ctx_refines CellImpl CellSpec ⊢  
  refines (CellImpl ★ Client) (CellSpec ★ Client).
```

deployed program		proof model
CellImpl ★ Client	—— refines —→	CellSpec ★ Client

```
iIntros "CELL_REF".  
iSpecialize ("CELL_REF" $! Client).  
iExact "CELL_REF".
```

Both sides use the same `Client` module definition. They are separate executions with separate runtime states.

The client team proves against **CellSpec**

ClientSpec describes the externally visible behavior of the application.

```
Check (⊢ ctx_refines  
  (CellSpec ★ Client)  
  (CellSpec ★ ClientSpec)).
```

```
CellSpec ★ Client —client proof→ CellSpec ★ ClientSpec  
  unchanged contract                abstraction
```

The replacement remains open for linking

```
ctx_refines CellImpl CellSpec ⊢  
  ctx_refines (CellImpl ★ Client) (CellSpec ★ Client)
```

TARGET		SOURCE
CellImpl ★ Client	←refinement proof→	CellSpec ★ Client
replacement		contract + same Client

Deployment uses `CellImpl` ; the proof carries its traces back to `CellSpec` .

The modular replacement proof

```
Lemma replace_cell_in_client :  
  ctx_refines CellImpl CellSpec ⊢  
  ctx_refines (CellImpl ★ Client) (CellSpec ★ Client).
```

Proof.

```
  iIntros "CELL_REF".  
  jStartProof (ctx_refines_BiProset).  
  jIntros "(IMPL & CLIENT)".  
  jPoseProof "CELL_REF" with "IMPL" as "SPEC".  
  jFrame.
```

Qed.

Compose the provider and client proofs

```
ctx_refines CellImpl CellSpec *  
  ctx_refines (CellSpec ★ Client) (CellSpec ★ ClientSpec) ⊢  
ctx_refines (CellImpl ★ Client) (CellSpec ★ ClientSpec)
```

```
CellImpl ★ Client —replacement→ CellSpec ★ Client  
                        | client proof  
                        ▼  
                    CellSpec ★ ClientSpec
```

```
first edge, Ctx := ⊙ : refines (CellImpl ★ Client)  
                        (CellSpec ★ Client)  
composed,   Ctx := ⊙ : refines (CellImpl ★ Client)  
                        (CellSpec ★ ClientSpec)
```

Adequacy closes this concrete pair

```
Mt := CellImpl ★ Client      Ms := CellSpec ★ ClientSpec
```

```
refines Mt Ms  
  | refines_adequacy  
  | requires Mod.wf Mt + initial world invariant
```

▼

```
∃ rs, ✓ rs ∧  
  refines_lmod (Mod.to_lmod Mt ε) (Mod.to_lmod Ms rs)  
  | definition
```

▼

```
Beh.of_itree (compile (Mod.to_lmod Mt ε))  
  ⊆  
Beh.of_itree (compile (Mod.to_lmod Ms rs))
```


The result is behavior inclusion

```
trace ∈ Beh(compile (Mod.to_lmod Mt ε))  
      | refines_lmod  
      ▼  
trace ∈ Beh(compile (Mod.to_lmod Ms rs))
```

```
Lemma refinement_means_behavior_inclusion  
  (Target Source : LMod.t) (trace : Tr.t) :  
  refines_lmod Target Source ->  
  Beh.of_itree (LMod.compile Target tt↑) trace ->  
  Beh.of_itree (LMod.compile Source tt↑) trace.
```

Every behavior produced by the target layer is permitted by the source layer.

Why this is program verification

Suppose every source behavior satisfies P .

```
trace ∈ Beh(Target)
      | refinement
      ▼
trace ∈ Beh(Source)
      | SOURCE_OK
      ▼
    P trace
```

Examples of P : output safety, a return-value postcondition, or protocol compliance.

Rocq makes property transfer explicit

```
Lemma refinement_transfers_universal_properties
  (Target Source : LMod.t) (P : Tr.t -> Prop)
  (REFINE : refines_lmod Target Source)
  (SOURCE_OK : forall trace,
    Beh.of_itree (LMod.compile Source tt↑) trace ->
    P trace) :
  forall trace,
    Beh.of_itree (LMod.compile Target tt↑) trace ->
    P trace.

Proof.
  intros trace TARGET_BEHAVIOR.
  apply SOURCE_OK.
  apply REFINE.
  exact TARGET_BEHAVIOR.
Qed.
```

The next goal is contextual refinement

For a source module and its replacement target, we want:

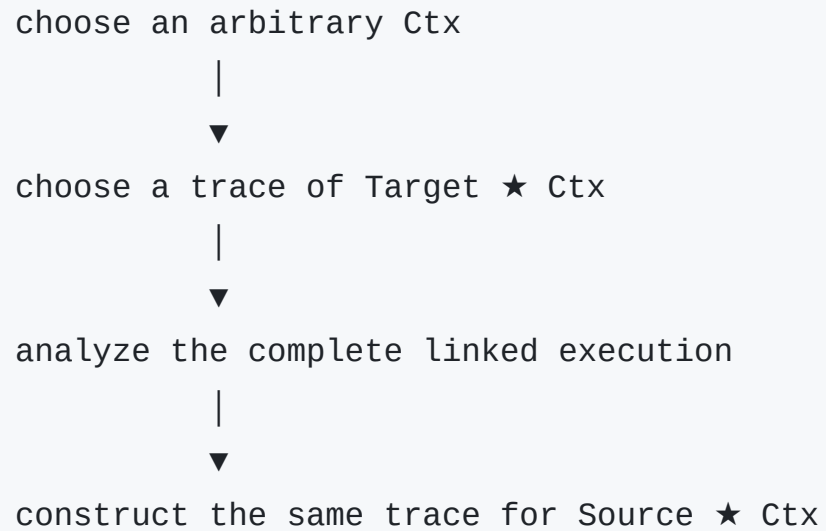
```
⊢ ctx_refines Target Source
```

Unfolding the outer definition exposes the quantification:

```
for every compatible Ctx,  
  Beh(Target ★ Ctx) ⊆ Beh(Source ★ Ctx)
```

We need a reusable proof technique for this all-context behavior claim.

Whole-execution reasoning scales poorly



Changing `ctx` changes the execution tree. Calls may re-enter the module, and the trace may be finite, blocked on I/O, or infinite.

The local reason that the two modules correspond is buried inside this global argument.

Simulation is the local proof technique

A simulation relates pairs of execution configurations.



The proof follows the programs one step at a time and re-establishes R .

The simulation rule preserves observations

RELATED CONFIGURATIONS

- |
- | $\vdash$ internal step $\rightarrow$ match with internal source steps
- |
- | $\vdash$ I/O event $\longrightarrow$ expose the same request and shared response
- |
- | $\vdash$ Call $f(x)$ $\longrightarrow$ match the same function name and argument
- |
- | $\vdash$ Return v $\longrightarrow$ return the same value

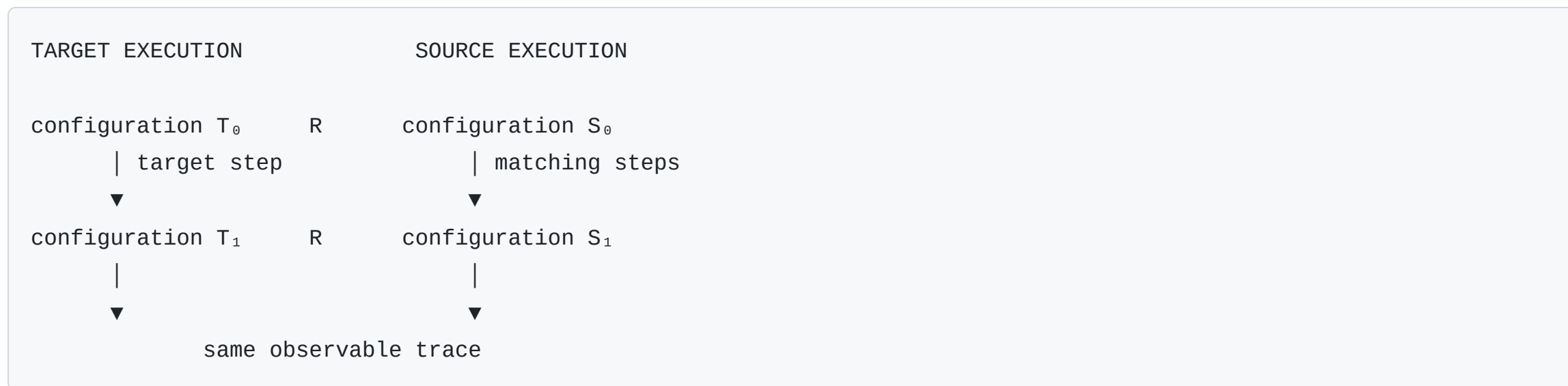
|



RELATED CONTINUATIONS

Each rule matches the currently visible interaction and supplies the relation needed to continue.

Repeated matching produces a source behavior



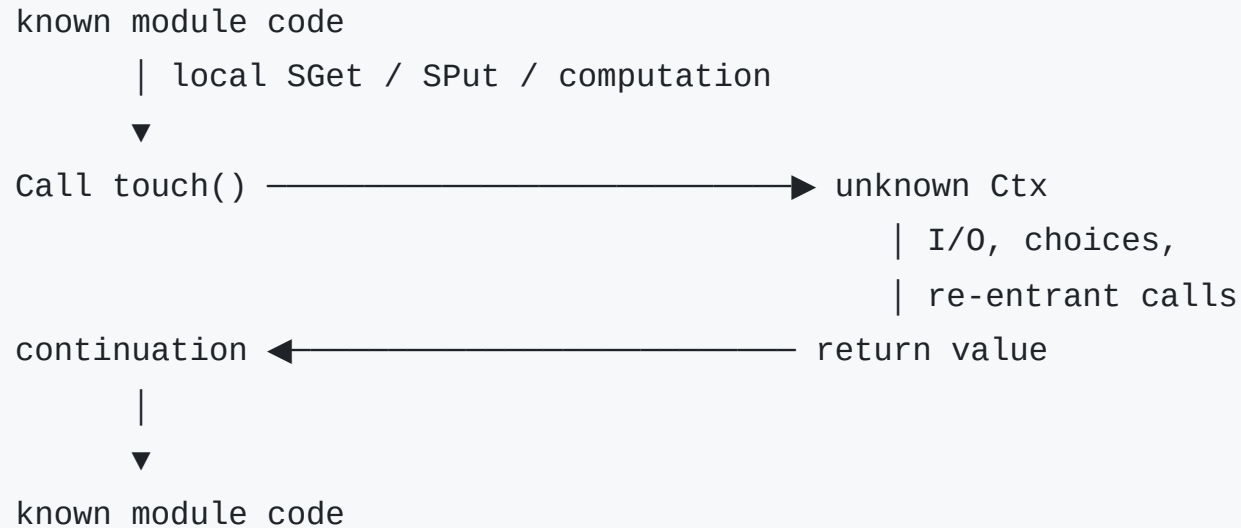
Finite executions finish with matching returns or I/O boundaries.

Infinite executions continue the rule coinductively.

CRIS packages this soundness argument as:

```
ISim.t open Source Target emp Ist -> ⊢ ctx_refines Target Source
```


Modules create control boundaries



The simulation proof handles execution while the module has control.

The linking context handles execution between an unknown call and its return.

A simulation relation must support both transfers of control.

Simulation needs a relation on local states

SOURCE CONFIGURATION

TARGET CONFIGURATION

$\langle \text{source state, source code} \rangle \quad R \quad \langle \text{target state, target code} \rangle$

The remaining computations determine which facts the relation must retain.

before local steps	enough information to match the next steps
before unknown Call	enough information to transfer control
after Call	enough information to resume both continuations
at return	related final states

Source and target states may use different representations.

CRIS names the state relation **Ist**

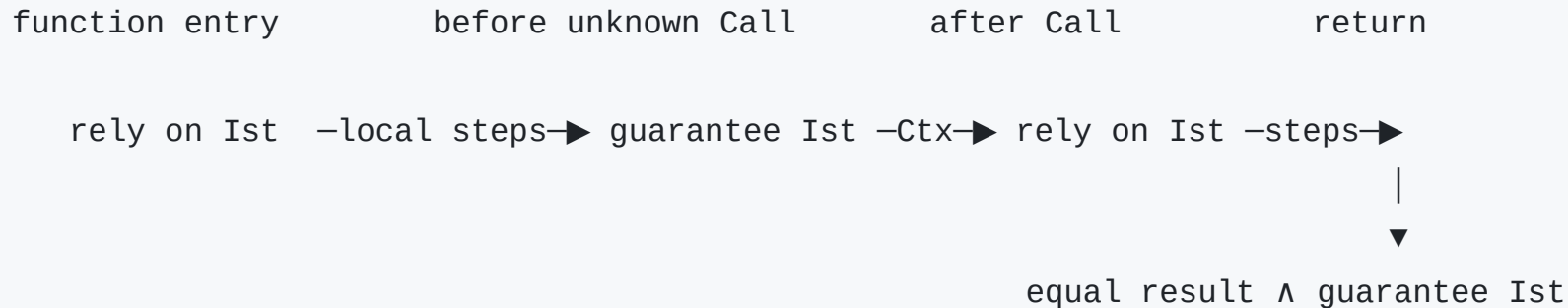
The exact type is:

```
ist_type  $\Sigma$  :=  
  gmap key (option Any.t) ->  
  gmap key (option Any.t) ->  
  iProp  $\Sigma$ 
```

```
Ist source_state target_state
```

Ist may express **True**, equality of selected state entries, or a representation invariant.

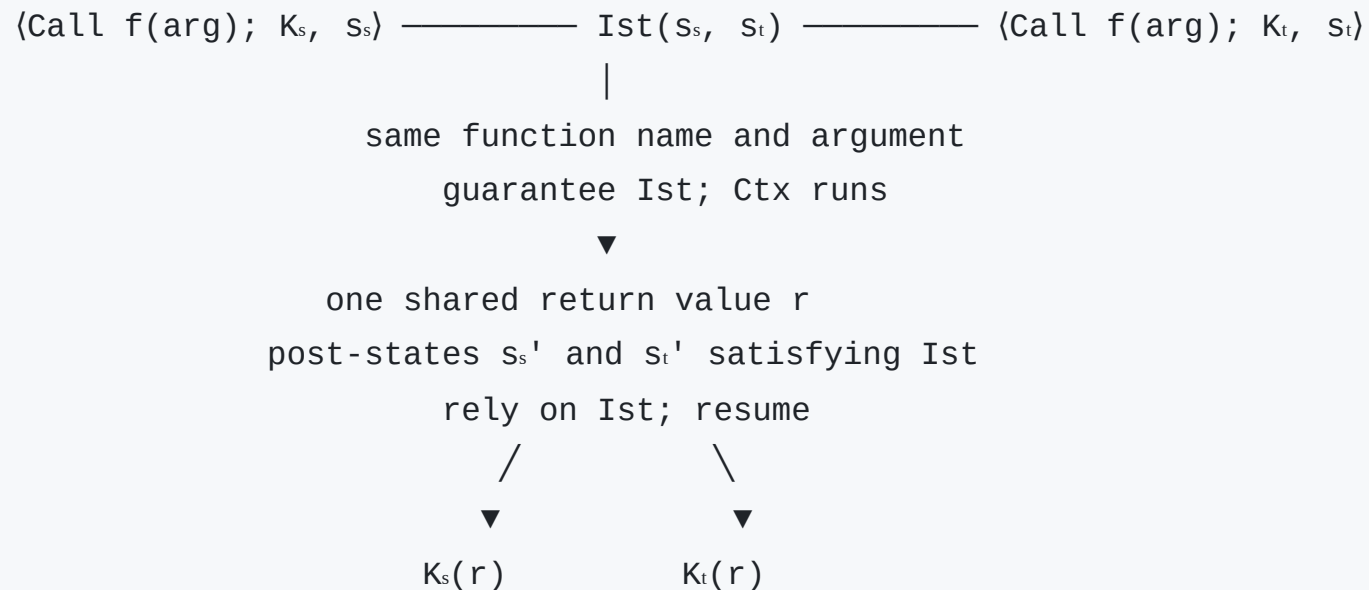
Ist is the control-boundary contract



- Known function code may assume **Ist** when it receives control.
- Before releasing control, it establishes **Ist**.
- A synchronized return also establishes equal results.

Every exported function obeys the same contract, so a re-entrant call starts from **Ist**.

The unknown-Call rule uses the boundary contract



The simulation proof summarizes the context execution through its return value and the restored `Ist` .

The per-function simulation statement

For each exported function name `fn`, prove:

```
ISim.sim_fun open Source Target Ist fn
```

The statement requires the simulation to:

```
receive source and target entry states satisfying Ist  
match the two function bodies under the simulation rules  
match observable events and unresolved calls  
return equal values with final states satisfying Ist
```

Formal chain · assemble the module simulation

```
Lemma simF_fn :  
  ISim.sim_fun open Source Target Ist fn.  
  
Lemma sim : ISim.t open Source Target emp%I Ist.  
Proof.  
  cStartModSim.      (* initial Ist + one goal per exported function *)  
  - (* prove the initial states satisfy Ist *)  
  - apply simF_fn.    (* repeat for every exported function *)  
Qed.
```

cStartModSim turns the module statement into:

```
source function-map well-formedness  
initial source and target states satisfy Ist  
one ISim.sim_fun goal for each exported source function
```

Formal chain · obtain behavior inclusion

Lemma `ctxr` : $\vdash$ `ctx_refines` `Target` `Source`.

Proof. `eapply` `main_adequacy`, `sim`. `Qed`.

```
ctxr
≡ ∀ Ctx, refines (Target ★ Ctx) (Source ★ Ctx)
      | refines_adequacy
      | requires Mod.wf (Target ★ Ctx)
      | + initial world invariant
      ▼
Mt := Mod.to_lmod (Target ★ Ctx) ε; Ms(rs) := Mod.to_lmod (Source ★ Ctx) rs

∃ rs, ✓ rs ∧ refines_lmod Mt (Ms rs)
      | definition of refines_lmod
      ▼
Beh.of_itree (LMod.compile Mt tt↑)
  ⊆
Beh.of_itree (LMod.compile (Ms rs) tt↑)
```


A simulation proof, one goal at a time

Start with empty local state.

```
Source: return (1 + 2)
```

```
Target: return 3
```

Open `day1/exercises/Optimizations.v`.

One interface, two bodies

```
(* Source *)  
fun _ => Ret ((1 + 2)%Z)  
  
(* Target *)  
fun _ => Ret 3%Z
```

The target has performed constant folding.

The first state relation is True

```
Definition Ist : ist_type  $\Sigma$  :=  
  fun _ _ => True%I.
```

Both modules start with empty local state.

Every pair of local states satisfies this first relation.

Goal 1 · Simulate one function

```
Lemma simF_fold_constants :  
  ISim.sim_fun open Source Target Ist  
    (fid StatelessOptHdr.fold_constants).
```

```
one exported function  
  source body ~ target body
```

Enter the function bodies

```
cStartTypedFunSim u.
```

The generated goal contains:

```
"IST" : Ist st_src st_tgt
-----
source computation
target computation
postcondition: ist_with_eq Ist
```

`cStartTypedFunSim` resolves function lookup, decodes `Any.t`, and exposes the typed argument `u : ()`.

Expose the two returns

```
unfold StatelessOptSource.fold_constants,  
       StatelessOptTarget.fold_constants.
```

Read the center of the generated `wsim` goal as:

```
source: Ret (1 + 2)  
target: Ret 3  
post:   ist_with_eq Ist
```

Match the two returns

```
cStep.
```

Rocq now asks for:

```
ist_with_eq Ist  
  (st_src, (1 + 2)%Z↑)  
  (st_tgt, 3%Z↑)
```

Its definition is:

```
⌈(1 + 2)%Z↑ = 3%Z↑⌋ * Ist st_src st_tgt
```

GUIDED DEMO 1 · Finish the function

Continue from the `ist_with_eq` goal:

```
iSplit; eauto.
```

Read `iSplit` from the connective in the goal:

```
P * Q
|   └─ Ist st_src st_tgt, which reduces to True
└─── equal return values
```

Type the proof with the instructor, then replace `Admitted` with `Qed`.

The optimization crosses an I/O boundary

```
(* Source *)
x <- trigger (IO (I := Z) "input" tt);;
Ret (x + 0)%Z

(* Target *)
x <- trigger (IO (I := Z) "input" tt);;
Ret x
```

`Behavior.v` records the external request and reply in the trace.

The optimization preserves that event and simplifies its continuation.

Match the event, then prove every continuation

```
source: IO "input" () —reply→ return (reply + 0)
target: IO "input" () —reply→ return reply
```

```
cStep as reply.
cStep.
iSplit;
  [replace (reply + 0)%Z with reply by lia; done | done].
```

`reply` is arbitrary. One I/O simulation step gives both continuations the same environment response. `lia` discharges the integer identity $\text{reply} + 0 = \text{reply}$.

A loop creates a family of simulation goals

```
(* Source *)
ITree.iter
  (fun cursor =>
    match cursor with
    | 0 => Ret (inr 0)
    | S cursor' => Ret (inl cursor')
  end)
n

(* Target *)
Ret 0
```

The source takes `n` silent iterations. A proof for arbitrary `n` follows the decreasing loop cursor.

One unfolding exposes a smaller loop

Here `body` is the cursor function shown on the preceding slide.

```
ITree.iter body 0
  = Ret 0

ITree.iter body (S n)
  = Tau (ITree.iter body n)
          ▲
      induction hypothesis
```

```
iInduction n as [|n] "IH".
aUnfoldS.
```

Successor case: `aUnfoldS` exposes `Tau` and the smaller iterator.

STOP 1 · Follow the countdown cursor

Complete `simF_countdown` .

The starter has already executed:

```
iInduction n as [|n] "IH"; aUnfoldS.
```

Finish the two visible goals:

```
- cStep. iSplit; eauto.  
- cStepS. iApply ("IH" with "IST").
```

Replace `Admitted` with your proof and `Qed` .

Reference solution: `day1/answers/Optimizations.v`

Goal 2 · Assemble three function simulations

```
Lemma sim : ISim.t open Source Target emp%I Ist.  
Proof.  
  cStartModSim.
```

The module proof checks:

```
source function-map well-formedness  
empty initial states satisfy Ist = True  
simF_fold_constants  
simF_read_simplify  
simF_countdown
```

Register the three per-function simulations

```
all: try solve [mod_tac].  
- apply simF_fold_constants.  
- apply simF_read_simplify.  
- apply simF_countdown.  
Qed.
```

```
return + I/O + finite loop simulations  
      ↓  
    one module simulation
```

Goal 3 · Obtain contextual refinement

```
Lemma ctxr :  $\vdash$  ctx_refines Target Source.
```

```
Proof.
```

```
  eapply main_adequacy, sim.
```

```
Qed.
```


Recap · `ISim.t` proves contextual refinement

For every refinement example in this workshop:

```
per-function statements
ISim.sim_fun open Source Target Ist fn
    | cStartModSim + apply
    ▼
module-level statement
ISim.t open Source Target emp%I Ist
    | main_adequacy
    ▼
⊢ ctx_refines Target Source
```

Lemma `ctxr` : $\vdash \text{ctx_refines Target Source}$.

Proof. `eapply` `main_adequacy`, `sim`. `Qed`.

Store-to-load forwarding

```
(* Source *)  
cput cell x;;;  
y <- cgetU cell;;  
Ret y
```

```
(* Target *)  
cput cell x;;;  
Ret x
```

Both stores are internal module steps.

The earlier relation, now in Rocq

```
Definition Ist : ist_type  $\Sigma$  :=  
  fun st_src st_tgt =>  
    ( $\exists$  x : Z,  
       $\ulcorner$ st_src = {[StateOptSource.cell # x $\uparrow$ ]}  $\wedge$   
        st_tgt = {[StateOptTarget.cell # x $\uparrow$ ]} $\urcorner$ )%I.
```

source cell: x ~ target cell: x

This is the `Ist` used in the paired callback diagrams.

`StateOptHdr.mn ↪ "cell"` creates a state key qualified by the module name.

Read proof-mode tactics from connectives

Assertion shape	Proof action
<code>⌈P⌋</code>	<code>iPureIntro</code> moves <code>P</code> to Rocq
<code>P * Q</code>	<code>iSplit</code> creates one goal for each part
<code>∃ x, P x</code>	<code>iExists v</code> chooses the witness
<code>"H" : ∃ x, ⌈P x⌋</code>	<code>iDestruct "H" as (x) "%"</code>

The state relation has the final shape:

```
∃ x : Z, ⌈source cell = x /\ target cell = x⌋
```

Prove the re-entry function first

```
initial relation      (old, old)
source store         (x,   old)
target store         (x,   x)
new relation witness  x
```

```
iDestruct "IST" as (old) "%".
cStepsS. cStepsT.
```

`cStepsS` and `cStepsT` execute consecutive deterministic steps on the source and target sides.

STOP 2 · Restore the cell relation

Complete `simF_store_load` .

The starter has matched the returns and left the new `Ist` goal:

```
iExists x. done.
```

Replace `Admitted` with your proof and `Qed` .

Reference solution: `day1/answers/Optimizations.v`

Return to the callback diagram

Both bodies write `x`, call a function supplied by the linking context, then read their cells:

```
cput cell x;;  
ccallU CallbackHdr.touch tt;;  
y <- cgetU cell;;  
Ret (y + 0)%Z          (* source *)  
Ret y                  (* target *)
```

The context may call an exported function before `touch` returns. In the two paired executions, that call can update the corresponding source-side and target-side cells. The final logical value can therefore differ from `x`.

Encode the boundary guarantee

Before the call, establish the guarantee:

```
iAssert  
  (Ist {[StateOptSource.cell # x↑]}  
      {[StateOptTarget.cell # x↑]}})  
  as "IST".  
{ iExists x. done. }
```

`iAssert P as "H"` proves an intermediate assertion and keeps it under the name `"H"`.

Match the unknown call:

```
cCall "IST" as (ret st_src st_tgt) "IST".
```


Encode the post-call rely

After the call:

<code>ret</code>	one arbitrary return shared by both continuations
<code>st_src, st_tgt</code>	an arbitrary pair of post-call states
<code>"IST"</code>	constraint: <code>Ist st_src st_tgt</code>

The earlier context, now in Rocq

```
Definition touch : () -> itree crisE () :=  
  fun _ =>  
    ccallU StateOptHdr.store_load 7%Z;;;  
    Ret tt.
```

```
callback_read x  
  ⊢ touch()  
    ⊢ store_load 7  
  ⊢ read cell = 7
```

The final logical cell value is **7** in both paired executions.

Specialize the all-context theorem

The contextual theorem already covers `ReenteringContext.t` :

```
⊢ refines  
  (Target ★ ReenteringContext.t)  
  (Source ★ ReenteringContext.t)
```

```
iPoseProof ctxr as "REF".  
iSpecialize ("REF" $! ReenteringContext.t).  
iExact "REF".
```

- `iPoseProof` names the reusable contextual theorem.
- `iSpecialize` chooses this concrete context.
- `iExact` closes the matching refinement goal.

First non-trivial data-representation proof

SOURCE STATE		TARGET STATE
abstract key-value map	~	sorted list of key-value pairs

This example defines an `Ist` that connects two different representations of the same logical storage.

Open `day1/exercises/KVSortedList.v`.

day1/exercises/KVSortedList.v

One API, two private representations

```
get : Z → option Z  
put : Z × Z → unit
```

```
Source state : Z → option Z  
Target state : list (Z × Z)
```

Clients see the API. The module owns the representation.

Source lookup is immediate

```
Definition get : Z -> itree crisE (option Z) :=  
  fun k =>  
    'm : amap <- cgetU v_map;;  
    Ret (m k).
```

```
Definition put : Z * Z -> itree crisE () :=  
  ...  
  cput v_map (map_put m k v);;;  
  Ret tt.
```

Target state is a sorted mathematical list

Definition `entries := list (Z * Z).`

Fixpoint `list_get (q : Z) (xs : entries) : option Z := ...`

Fixpoint `list_put (k v : Z) (xs : entries) : entries := ...`

- keys are strictly increasing
- equal keys are overwritten
- lookup stops once the head key exceeds the query

The representation relation

```
Definition state_rel (m : amap) (xs : entries) : Prop :=  
  sorted_keys xs /\  
  forall k, m k = list_get k xs.
```

sortedness	supports ordered insertion and early lookup exit
lookup agreement	connects every observable get result

Provided algebra for updates

```
sorted_keys_put :  
  sorted_keys xs ->  
  sorted_keys (list_put k v xs)  
  
list_get_put :  
  list_get q (list_put k v xs) =  
  map_put (fun q => list_get q xs) k v q
```

STOP 3 · Relate the empty states

Complete `state_rel_empty`.

```
empty_map ~ []
```

The starter has already unfolded `state_rel` and split its conjunction:

- `constructor`.
- `reflexivity`.

Reference solution: `day1/answers/KVSortedList.v`

STOP 4 · Preserve the relation under `put`

Complete `state_rel_put`. The starter has exposed its two conjuncts:

```
- apply sorted_keys_put. exact Hsorted.  
- intros q.  
  rewrite list_get_put. unfold map_put.  
  destruct (Z.eq_dec q k);  
    [reflexivity | apply Hlookup].
```

Reference solution: `day1/answers/KVSortedList.v`

From `state_rel` to the full `Ist`

`IstLocal` owns the map/list cells and stores `state_rel m xs`.

The full relation also tracks state owned by the linking context:

```
Ist =  
  IstProd  
    (IstSB Source.scopes IstLocal)  
    IstEq
```

A provided helper hides this packaging:

```
Ist_from_state_rel m xs st :  
  「state_rel m xs」 ⊢  
  Ist  
    (union_with uwnd {[KVSource.v_map # m↑]} st)  
    (union_with uwnd {[SortedListTarget.v_entries # xs↑]} st)
```

Put repeats the restoration pattern

```
pre-state      m      ~  xs
source update  map_put m k v
target update  list_put k v xs
post-state     related by state_rel_put
```

```
pose proof (state_rel_put m xs k v Hrel) as Hupdated.
cStep.
iSplit; [eauto |].
```

STOP 5 · KV put simulation

Complete `simF_put`.

```
iApply (Ist_from_state_rel _ _ st_tgtR).  
iPureIntro.  
exact Hupdated.
```

`Hupdated` is the mathematical step. The provided helper rebuilds the CRIS state relation.

Reference solution: `day1/answers/KVSortedList.v`

Lookup carries a cursor

```
Definition get : Z -> itree crisE (option Z) :=
  fun q =>
    'xs : entries <- cgetU v_entries;;
    ITree.iter
      (fun rest : entries =>
        match rest with
        | [] => Ret (inr None)
        | (k, v) :: tl =>
          match Z.compare q k with
          | Lt => Ret (inr None)
          | Eq => Ret (inr (Some v))
          | Gt => Ret (inl tl)
        end
      end)
    xs.
```

One skipped head produces Tau

```
match Z.compare q k with
| Lt => Ret (inr None)
| Eq => Ret (inr (Some v))
| Gt => Ret (inl tl)
end
```

`ITree.iter` interprets `inl tl` as continuation with the tail.

Gt branch: one silent step, then search `tl`

The proof follows the cursor

```
pose proof Hrel as Hstate.  
destruct Hrel as [_ Hlookup]. rewrite Hlookup.  
generalize xs at 1 3. iIntros (cursor).
```

After the rewrite, the goal contains the same list in two computational positions: the source lookup and the target iterator cursor.

`generalize xs at 1 3` replaces those occurrences with one `cursor`.

`iIntros (cursor)` introduces that quantified value into the Iris goal.

`Hstate` keeps the relation for restoring `Ist`.

Unfold one target iteration

```
iInduction cursor as [| [k' v'] tl] "IH"; aUnfoldT.
```

This first creates `[]` and nonempty-list goals. In the nonempty goal:

```
simpl. destruct (Z.compare k k') eqn:Hcmp.
```

Cursor case	Proof action
<code>[]</code>	<code>cStep</code> , then restore <code>Ist</code>
<code>Eq</code>	<code>cStep</code> , then restore <code>Ist</code>
<code>Lt</code>	<code>cStep</code> , then restore <code>Ist</code>
<code>Gt</code>	<code>cStepT</code> ; <code>iApply "IH"</code>

`aUnfoldT` exposes one iteration; `cStepT` consumes the `Gt` -branch `Tau` .

STOP 6 · Iterator induction

Complete `simF_get` .

Continue after the provided `iInduction` and `aUnfoldT` .

- `[]` : use `cStep` , `iSplit` , and `Ist_from_state_rel` .
- `Nonempty`: run `simpl`; destruct `(Z.compare k k')` `eqn:Hcmp` .
- `Eq` / `Lt` : reuse the terminating-case pattern.
- `Gt` : use `cStepT`; `iApply "IH"` .

Reference solution: `day1/answers/KVSortedList.v`

Assemble and return to behaviors

Lemma `sim : ISim.t open Source Target emp%I Ist.`

Lemma `ctxr : ⊢ ctx_refines Target Source.`

`state_rel_empty`

|

`simF_put + simF_get`

|

▼

`module simulation`

|

`main_adequacy`

▼

`ctx_refines SortedListTarget KVSource`

Optional advanced · Make `put` iterative

```
cursor = (prefix, rest)
```

<code>rest = []</code>	→ finish with <code>prefix ++ [(k, v)]</code>
<code>k < head.key</code>	→ finish with <code>prefix ++ (k, v) :: rest</code>
<code>k = head.key</code>	→ finish with <code>prefix ++ (k, v) :: tail</code>
<code>k > head.key</code>	→ continue with <code>(prefix ++ [head], tail)</code>

Maintain this equation at every iterator step:

```
prefix ++ list_put k v rest = list_put k v original
```

Open `day1/exercises/KVSortedListAdvanced.v`.

Back to the first question

sorted-list target + any compatible client
| link and compile
▼
target behavior
⊆
source behavior

Every target behavior is permitted by the abstract key-value source.